

第八章 无失真的信源编码

第一节 霍夫曼(Huffman)码

第二节 费诺(Fano)码

第三节 香农-费诺-埃利斯码

***第四节 游程编码和MH编码**

第五节 算术编码

第六节 字典码

- 香农编码定理虽然指出了理想编码器的存在性,但是并没有给出实用码的结构及构造方法;
- 编码理论正是为了解决这一问题而发展起来的科学理论; 编码的目的是为了优化通信系统, 就是使这些指标达到最佳;
- 通信系统的性能指标主要是有效性、可靠性、安全性和经济性, 除了经济性外, 这些指标正是信息论研究的对象。
- 按不同的编码目的, 编码分为三类: **信源编码**、**信道编码**和**安全编码/密码**。

信源编码：以提高通信有效性为目的的编码。通常通过压缩信源的冗余度来实现。采用的一般方法是压缩每个信源符号的平比特数或信源的码率。即同样多的信息用较少的码率传送，使单位时间内传送的平均信息量增加，从而提高通信的有效性。

信道编码：是提高信息传输的可靠性为目的的编码。通过增加信源的冗余度来实现。采用的一般方法是增大码率/带宽。与信源编码正好相反。

密码：是以提高通信系统的安全性为目的的编码。通常通过加密和解密来实现。从信息论的观点出发，“加密”可视为增熵的过程，“解密”可视为减熵的过程。

信源编码理论是信息论的一个重要分支，其理论基础是信源编码的两个定理。

无失真信源编码定理：是离散信源/数字信号编码的基础；

限失真信源编码定理：是连续信源/模拟信号编码的基础。

信源编码的分类：离散信源编码、连续信源编码和相关信源编码三类。

离散信源编码：独立信源编码，可做到无失真编码；

连续信源编码：独立信源编码，只能做到限失真信源编码；

相关信源编码：非独立信源编码。

无失真信源编码和限失真信源编码的应用

- 前者主要用于离散信源或数字信号，如文本、表格及工程图纸等信源，要求进行无失真的数据压缩，要求能够无失真地可逆恢复；
- 后者主要用于波形信源或波形信号，如语音、电视图像、彩色静止图像等信号，它们不要求完全可逆地恢复，而是允许在一定限度可以有失真的压缩。
- 两者目的都是为了用较少的码率来传送同样多的信息，提高系统有效性。

香农第一定理给出了**信源熵**与编码后的**平均码长**之间的关系，同时也指出可通过编码使平均码长达到极限值，因此，香农第一定理是一个极限定理。

但定理中并没有告诉我们如何来构造这种码。

最佳变长码：凡是能载荷一定的信息量，且码字的平均长度最短，可分离的变长码的码字集合称为最佳变长码。

8.1 霍夫曼码

香农编码

1、码长 $l_i = \left\lceil \log \frac{1}{P(s_i)} \right\rceil \quad (i = 1, 2, \dots, q)$

2、按照算出的码长、用树图法编出相应的即时码。

一般香农编码的 $\bar{L}$ 不是最短的，即不是最佳码。

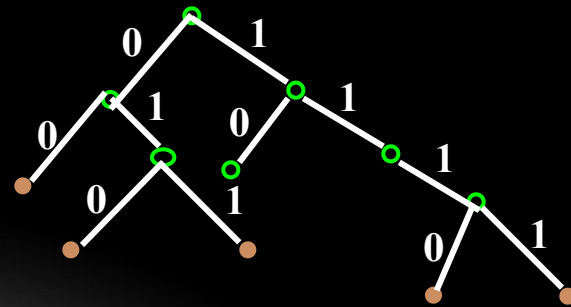
例8.1 用香农编码方法编成二元惟一可译码.

S	s_1	s_2	s_3	s_4	s_5
$P(s_i)$	0.4	0.2	0.2	0.1	0.1
$\log \frac{1}{P(s_i)}$	1.32	2.3	2.3	3.3	3.3
l_i	2	3	3	4	4

00 010 011 1110 1111 编码

$$\begin{aligned}\bar{L} &= 2 \times 0.4 + 3 \times 0.4 + 4 \times 0.2 \\ &= 2.8 \quad (\text{码元/信源符号})\end{aligned}$$

$$H(S) = 2.122 \quad \eta = \frac{H(S)}{\bar{L}} = 0.758$$



$$\frac{H(S)}{\log r} < \bar{L} < \frac{H(S)}{\log r} + 1$$

8.1.1 二元霍夫曼码

1952年霍夫曼提出了一种构造最佳码的方法。它是一种最佳的逐个符号的编码方法。其编码步骤如下：

(1) 将信源消息符号按其出现的概率大小依次排列

$$p(x_1) \geq p(x_2) \geq \cdots \geq p(x_q)$$

(2) 用“0”和“1”码符号分别代表概率最小的两个信源符号，并将这两个概率最小的符号相加合并成一个新的符号，新的符号概率为两个符号概率之和，然后与未分配的符号重新排队，从而得到只包含**q-1**个符号的新信源，称为**缩减信源**。

- (3) 重复步骤(2): 把缩减信源的符号仍旧按概率大小以递减次序排列, 再将其概率最小的两个信源符号分别用“0”和“1”表示, 并将其合并成一个符号, 概率为两符号概率之和, 这样又形成了 $q-2$ 个符号的缩减信源。
- (4) 不断继续上述过程, 直至信源只剩下两个符号为止。将这最后两个信源符号分别用“0”和“1”表示。
- (5) 然后从最后一级缩减信源开始, 向前返回(逆向), 就得出各信源符号所对应的码符号序列, 即对应的码字。

例8.1: 对离散无记忆信源 $\begin{bmatrix} S \\ p(s_i) \end{bmatrix} = \begin{bmatrix} s_1 & s_2 & s_3 & s_4 & s_5 \\ 0.4 & 0.2 & 0.2 & 0.1 & 0.1 \end{bmatrix}$ 进行霍夫曼编码。

解:编码过程如表所示,

- 1) 将信源符号按概率大小由大至小排序。
- 2) 从概率最小的两个信源符号和开始编码, 并按一定的规则赋予码符号, 如下面的信源符号 (小概率) 为 “1”, 上面的信源符号 (大概率) 为 “0”。若两支路概率相等, 仍为下面的信源符号为 “1” 上面的信源符号为 “0”。

- 3) 将已编码两个信源符号概率合并，重新排队，编码。
- 4) 重复步骤3) 直至合并概率等于“1.0”为止。
- 5) 从概率等于“1.0”端沿合并路线逆行至对应消息编码。

表 8.1 一种霍夫曼编码

信源符号 s_i	码字长度 l_i	码字 W_i	概率 $P(s_i)$	缩减信源		
				S_1	S_2	S_3
s_1			0.4	1 0.4	1 0.4	1 0.6 0.4 1
s_2			0.2	01 0.2	01 0.4	0 00
s_3			0.2	000 0.2	000 0.2	1 01
s_4			0.1	0010 0.2	001	
s_5			0.1	0011		

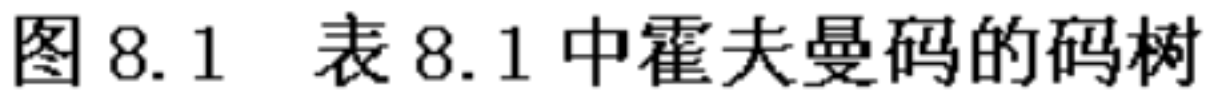
码的性能分析:

信源的熵为: $H(S) = 2.61$ (比特 / 符号)

可得平均码长为:

$$\bar{L} = \sum_{i=1}^5 p(s_i) l_i = 0.4 \times 1 + 0.2 \times 2 + 0.2 \times 3 + 0.1 \times 4 + 0.1 \times 4 = 2.2$$

$$\text{编码效率为: } \eta = \frac{H(S)}{\bar{L}} = 0.96$$



按霍夫曼码的编码方法，可知这种码有如下特征：

- ①它是一种分组码：各个信源符号都被映射成一组固定次序的码符号；
- ②它是一种惟一可解的码：任何码符号序列只能以一种方式译码；

③它是一种即时码：由于代表信源符号的节点都是终端节点，因此其编码不可能是其它终端节点对应的编码的前缀，霍夫曼编码所得的码字一定是即时码。所以一串码符号中的每个码字都可不考虑其后的符号直接解码出来。

霍夫曼码的译码：对接收到的霍夫曼码序列可通过从左到右检查各个符号进行译码。

说明:

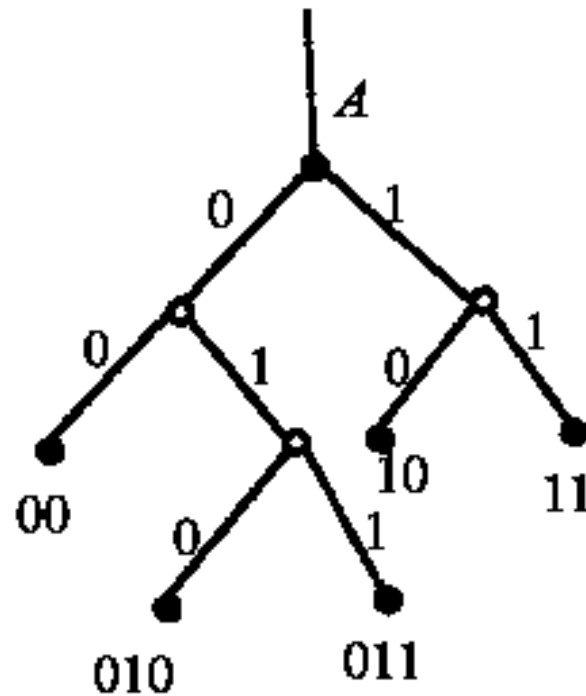
- ①霍夫曼码是一种即时码，可用码树形式来表示。
 - ②每次对缩减信源最后两个概率最小的符号，用“0”和“1”码是可以任意的，所以可得到不同的码，但码长不变，平均码长也不变。
 - ③当缩减信源中缩减合并后得到的新符号的概率与其他信源符号概率相同时，从编码方法上来说，它们概率的排序是没有限制的，因此也可得到不同的码。
- 即对给定信源，用霍夫曼编码方法得到的码并非是惟一，但平均码长不变。

表 8.2 另一种霍夫曼码

信源符号 s_i	码字长度 l_i	码字 W_i	概率 $P(s_i)$	缩减信源		
				S_1	S_2	S_3
s_1	2	00	0.4	00 → 0.4	00 → 0.4	00 → 0.6
s_2	2	10	0.2	10 → 0.2	01 → 0.2	01 → 0.4
s_3	2	11	0.2	11 → 0.2	10 → 0.2	11 → 0.4
s_4	3	010	0.1	0 010 → 0.2	1 11	
s_5	3	011	0.1	1 011		

可得平均码长为:

$$\bar{L} = \sum_{i=1}^5 p(s_i) l_i = (0.4 + 0.2 + 0.2) \times 2 + (0.1 + 0.1) \times 3 = 2.2$$



■ 讨论：哪种方法更好？

- 定义码字长度的方差 σ^2 ：长度 k_i 与平均码长 $\bar{K}$ 之差的平方的数学期望，即

$$\sigma^2 = E[(k_i - \bar{K})^2] = \sum_{i=1}^n p(x_i)(k_i - \bar{K})^2$$

- 编法一码字长度方差：

$$\sigma_1^2 = 0.4(1 - 2.2)^2 + 0.2(2 - 2.2)^2 + 0.2(3 - 2.2)^2 + (0.1 + 0.1)(4 - 2.2)^2 = 1.36$$

- 编法二码字长度方差：

$$\sigma_2^2 = (0.4 + 0.2 + 0.2)(2 - 2.2)^2 + (0.1 + 0.1)(3 - 2.2)^2 = 0.16$$

- 可见：第二种编码方法的码长方差要小许多。意味着第二种编码方法的码长变化较小，比较接近于平均码长。
 - 第一种方法编出的5个码字有4种不同的码长；
 - 第二种方法编出的码长只有两种不同的码长；
 - 显然，第二种编码方法更简单、更容易实现，所以更好。

结论：在哈夫曼编码过程中，对缩减信源符号按概率由大到小的顺序重新排列时，应使合并后的新符号尽可能在靠前的位置，这样可使合并后的新符号重复编码次数减少，使短码得到充分利用。

从以上的编码实例中可以看出，霍夫曼编码具有以下特点：

- **(1)**霍夫曼编码方法保证了概率大的符号对应于短码，概率小的符号对应于长码，充分利用了短码；
- **(2)**每次缩减信源的最后两个码字总是前面各位码元相同、最后一位码元不同，从而保证了霍夫曼码是即时码。
- **(3)**每次缩减信源的最长两个码字有相同的码长。

这三个特点保证了所得的霍夫曼编码一定是最佳码。

8.1.2 r 元霍夫曼码

对二元霍夫曼码的编码方法同样可以推广到 r 元编码中。不同的只是每次把概率最小的个符号合并成一个新的信源符号，并分别用 $0, 1, \dots, r-1$ ，等码元表示。

为了使短码得到充分利用，使平均码长为最短，必须使最后一步的缩减信源有 r 个信源符号。

因此对于**r**元编码，信源**S**的符号个数**q**必须满足：

$$q = (r - 1)\theta + r$$

其中， θ 表示缩减的次数，**r-1**为每次缩减所减少的信源符号个数。

对于二元码（**r=2**），信源符号个数**q**必须满足：

$$q = \theta + 2$$

因此，**q**可等于任意整正数时，总能找到一个 θ 满足上式。

注意:

对于 r 元码时，不一定能找到 θ 使式 $q = (r-1)\theta + r$ 成立。在不满足上式时，可假设一些信源符号：

$s_{q+1}, s_{q+2}, \dots, s_{q+t}$ 作为虚拟的信源，并令它们对应的概率为零，即： $p_{q+1} = p_{q+2} = \dots = p_{q+t} = 0$

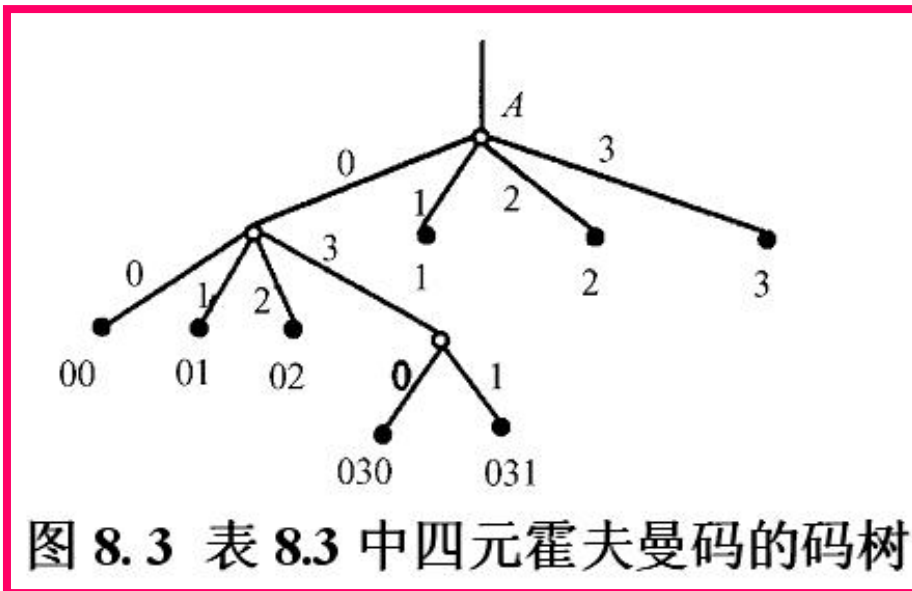
而使 $q + t = (r-1)\theta + r$ 能成立，这样处理后得到的 r 元霍夫曼码可充分利用短码，一定是紧致码。

例8.2 对信源进行四元霍夫曼码编码。表列出了四元霍夫曼码的编码方法及其对应的码字。

$$q = (r - 1)\theta + r$$

$$q + t = (r - 1)\theta + r$$

信源符号 s_i	码字 w_i	概率分布 $P(s_i)$	缩减信源		
			S_1	S_2	S_3
s_1	1	0.22	1 0.22	1 0.4	0 1 2 3
s_2	2	0.20	2 0.20	2 0.22	
s_3	3	0.18	3 0.18	3 0.2	
s_4	00	0.15	00 0.15	00 0.18	
s_5	01	0.10	01 0.10	01	
s_6	02	0.08	02 0.08	02	
s_7	030	0.05	03 0.07	03	
s_8	031	0.02	03		
s_9		0			
s_{10}		0			



这种编码方法是尽量把短码都利用上。首先，把一阶节点全都用上，如果码字不够时，再从某个节点伸出若干枝，引出二阶节点作为码字。再不够时，再伸出三阶节点。显然，这样得到的平均码长最短。

8.1.3 霍夫曼码的最佳性

定理 8.1 对于给定分布的任何信源,存在一个最佳二元即时码(其平均码长最短),此码满足以下性质:

- (1) 若 $p_j > p_k$, 则 $l_j \leq l_k$ 。
- (2) 两个最小概率的信源符号所对应的码字必具有相同的码长。
- (3) 两个最小概率的信源符号所对应的码字,其除最后一位码元不同外,前面各位码元都相同。

【证明】

(1) 若有即时码 C' , 其 $p_j > p_k$ 而 $l'_j \geq l'_k$ 。我们可以交换这两个码字,得码 C 。交换后码 C 中 $l_j = l'_k, l_k = l'_j$ 其余 $l_i = l'_i, i \neq j, k$, 则有

$$\bar{L}(C') - \bar{L}(C) = \sum p_i l'_i - \sum p_i l_i \quad (8.6)$$

$$= p_j l'_j + p_k l'_k - p_j l'_k - p_k l'_j \quad (8.7)$$

$$= (p_j - p_k)(l'_j - l'_k) \quad (8.8)$$

因为 $p_j > p_k > 0$, 又因 $l'_j \geq l'_k$, 所以

$$\begin{aligned} \bar{L}(C') - \bar{L}(C) &\geq 0 \\ \bar{L}(C) &\leq \bar{L}(C') \end{aligned} \quad (8.9)$$

可见,交换后码 C 的平均码长变短,并满足

$$p_j > p_k, l_j \leq l_k$$

所以,我们可以将码中所有码字进行交换,使其满足 $p_1 \geq p_2 \geq \dots \geq p_q$, 而 $l_1 \leq l_2 \leq \dots \leq l_q$ 。这样得到的码,平均码长变短。

(2) 由性质(1)已知,两个最小概率的信源符号其码长最长,有 $p_{q-1} \geq p_q$,一定 $l_{q-1} \leq l_q$ 。但若码 C 中 $l_{q-1} \neq l_q$,可以将这个最长码字的最后几位截去,使 $l_{q-1} = l_q$ 。这不但不影响即时码的前缀条件,而且使平均码长 $\bar{L}(C)$ 更为缩短,缩短了 $p_q(l_q - l_{q-1})$ 。

这样,两个最小概率的信源符号所对应码字的码长最长,而且相等。这时,所得码的平均码长最短。

(3) 根据性质(1)和(2),我们可以将一即时码 C' 通过交换、截枝整理,得到平均码长为最短的码 C 。这时,两个最小概率的信源符号所对应的码字长度最长且相等。从树图角度来看,它们应是在同一节点伸出两枝叉的两个节点上(二元树)。若不在同一节点伸出两枝叉的两个节点上,一定在同一阶树枝的其他节点上(同一阶的节点码长相同)。否则,就与性质(1)、(2)不一致了。这时,可以重新安排码字,使码字 W_{q-1} 和 W_q 在同一阶的同一节点伸出两枝叉的两个节点上。这样重新安排并不改变平均码长 $\bar{L}(C)$ 。因此得,这两个最小概率的信源符号所对应的码字,其除最后一位码元不同外,前面各位码元都相同。

综合上述,我们证得,若 $p_1 \geq p_2 \geq \dots \geq p_q$,则存在最佳即时码 C ,其码长满足 $l_1 \leq l_2 \leq \dots \leq l_{q-1} = l_q$,而且码字 W_{q-1} 和 W_q 只有最后一位码元不同。

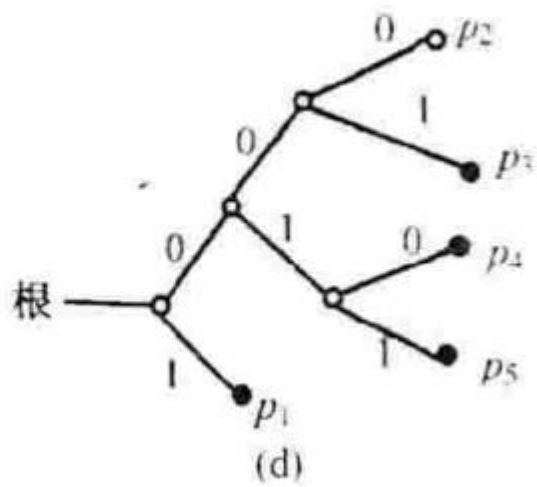
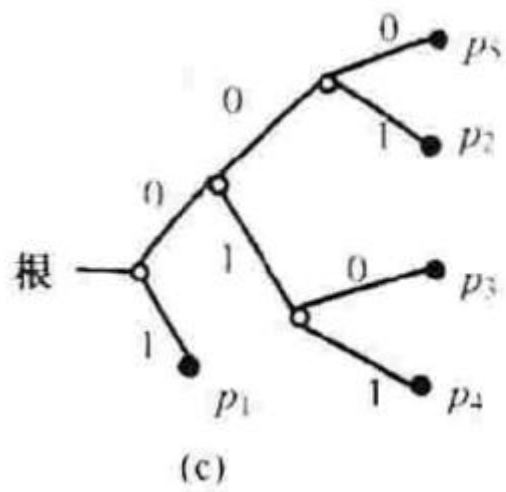
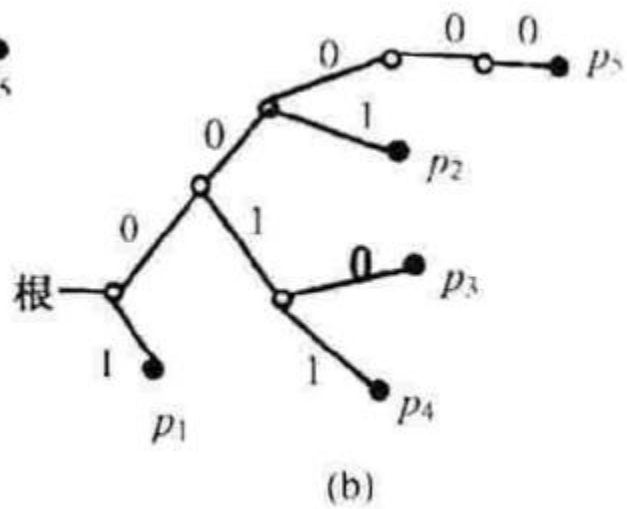
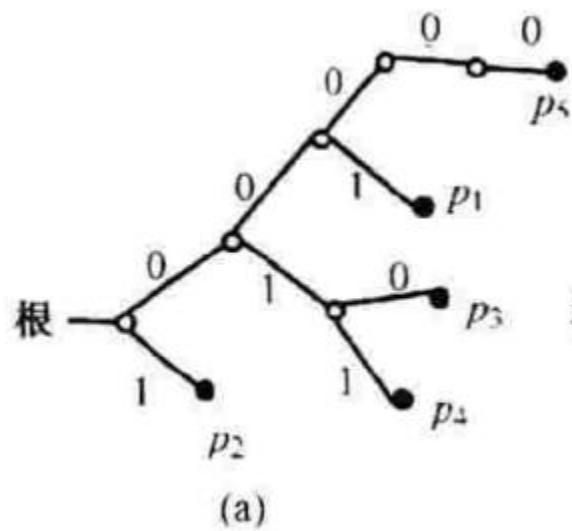


图 8.4 中假设 $P_1 \geq P_2 \geq P_3 \geq P_4 \geq P_5$ 。(a) 为某一即时码。(b) 交换 P_1, P_2 所得。(c) 将 P_3 截枝整理。(d) 重新安排得最佳二元码。

定理 8.2 二元霍夫曼码一定是最佳即时码。即若 C 是霍夫曼码, C' 是任意其他即时码, 则有

$$\bar{L}(C) \leq \bar{L}(C') \quad (8.11)$$

霍夫曼码是在信源给定情况下的最佳码, 所以其平均码长的界限为

$$H_r(S) \leq \bar{L}(C) < H_r(S) + 1 \quad (8.13)$$

8.2 费诺码

费诺编码属于概率匹配编码。它不是最佳的编码方法，只有当信源的概率分布呈现 $p(s_i) = r^{-l_i}$ 分布形式的条件下，才能达到最佳码的性能。

二元费诺码的编码步骤如下：

- 1) 信源符号以概率递减的次序排列起来；
- 2) 将排列好的信源符号按概率值划分成两大组，使每组的概率之和接近于相等，并对每组各赋予一个二元码符号“0”和“1”；
- 3) 将每一大组的信源符号再分成两组，使划分后的两个组的概率之和接近于相等，再分别赋予一个二元码符号；
- 4) 依次下去，直至每个小组只剩一个信源符号为止；
- 5) 信源符号所对应的码字即为费诺码。

例8.3 离散无记忆信源S,

表 8. 4 二元费诺码

信源符号	概率 $P(s_i)$		码字	码长 l_i
s_1	$1/4$	0	00	2
s_2	$1/4$		01	2
s_3	$1/8$	1	100	3
s_4	$1/8$		101	3
s_5	$1/16$	1	1100	4
s_6	$1/16$		1101	4
s_7	$1/16$	1	1110	4
s_8	$1/16$		1111	4

这种码是紧致码，码的效率等于1。之所以能达到最佳，是因为信源符号的概率分布正好满足式 $p(s_i) = r^{-l_i}$ (5.70)

例8.4 离散无记忆信源S,

表 8.5

信源符号	概率 $P(s_i)$		码字	码长 l_i
s_1	0.32	0	00	2
s_2	0.22		01	2
s_3	0.18	1	10	2
s_4	0.16		110	3
s_5	0.08	1	1110	4
s_6	0.04		1111	4

信源的熵为**2.35**比特/信源符号，码的平均码长为**2.32**，因此效率为**0.987**。

费诺码具有如下的性质：

①费诺码的编码方法实际上是一种构造码树的方法，所以费诺码是即时码。

②费诺码考虑了信源的统计特性，使概率大的信源符号能对应码长较短的码字，从而有效地提高了编码效率。

③费诺码不一定是最佳码。因为费诺码编码方法不一定能使短码得到充分利用:当信源符号较多时,若有一些符号概率分布很接近时,分两大组的组合方法就会很多。可能某种分大组的结果,会使后面小组的“概率和”相差较远,从而使平均码长增加。

- **r 元费诺码**

前面讨论的费诺码是二元费诺码,对r元费诺码,与二元费诺码编码方法相同,只是每次分组时应将符号分成概率分布接近的r个组。

评述

➤ 哈夫曼编码特点:

- ✓ 由于哈夫曼编码总是以最小概率相加的方法来“缩减”参与排队概率个数，因此概率越小，对缩减的贡献越大，其对于消息的码字也越长；
- ✓ 最小概率相加的方法使得编码不具有唯一性，尤其是碰到存在几个消息符号有着相同概率的情况，将会有多种路径选择，亦即具有多种可能的代码组集合；
- ✓ 尽管对同一信源存在着多种结果的哈夫曼编码，但它们的平均码长几乎都是相等的，因为每一种路径选择都是使用最小概率相加的方法，其实质都是遵循最佳编码的原则，因此哈夫曼编码是最佳编码。
- ✓ 哈夫曼编码是一种最佳编码，实现也不困难，因此到目前为止它仍是应用最为广泛的无失真信源编码之一。
- 在实际问题中真正使用哈夫曼编码时，还需要进一步研究有关误差扩散、速率匹配和概率匹配等问题。

评述

➤ 费诺编码特点:

- ✓ 概率大，则分解的次数小；概率小，则分解的次数多。这符合最佳编码原则。
- ✓ 码字集合是唯一的。
- ✓ 分解完了，码字出来了，码长也有了。
- ✓ 因此，费诺编码方法又称为子集分解法。
- ✓ 费诺编码方法比较适合于每次分组概率都很接近的信源，特别是对每次分组概率都相等的信源进行编码时，可达到理想的编码效率。

8.3 香农-费诺-埃利斯码

香农—费诺—埃利斯（Shannon—Fano—Elias）编码方法采用信源符号的累积分布函数来分配码字。

设信源符号集 $A = \{a_1, a_2, \dots, a_q\}$ ，并设所有 $P(a) > 0$ ， $a \in A$ ，定义累积分布函数为

$$F(s) = \sum_{a \leq s} P(a) \quad a \in A \quad (8.15)$$

也可写成

$$F(a_k) = \sum_{i=1}^k P(a_i) \quad a_k, a_i \in A \quad (8.16)$$

定义修正的累积分布函数：

$$\bar{F}(S) = \bar{F}(a_k) = \sum_{i=1}^{k-1} P(a_i) + \frac{1}{2}P(a_k) \quad a_k, a_i \in A \quad (8.17)$$

$\bar{F}(a_k)$ 值正处于对应 a_k 台级的中点，见图 8.5。

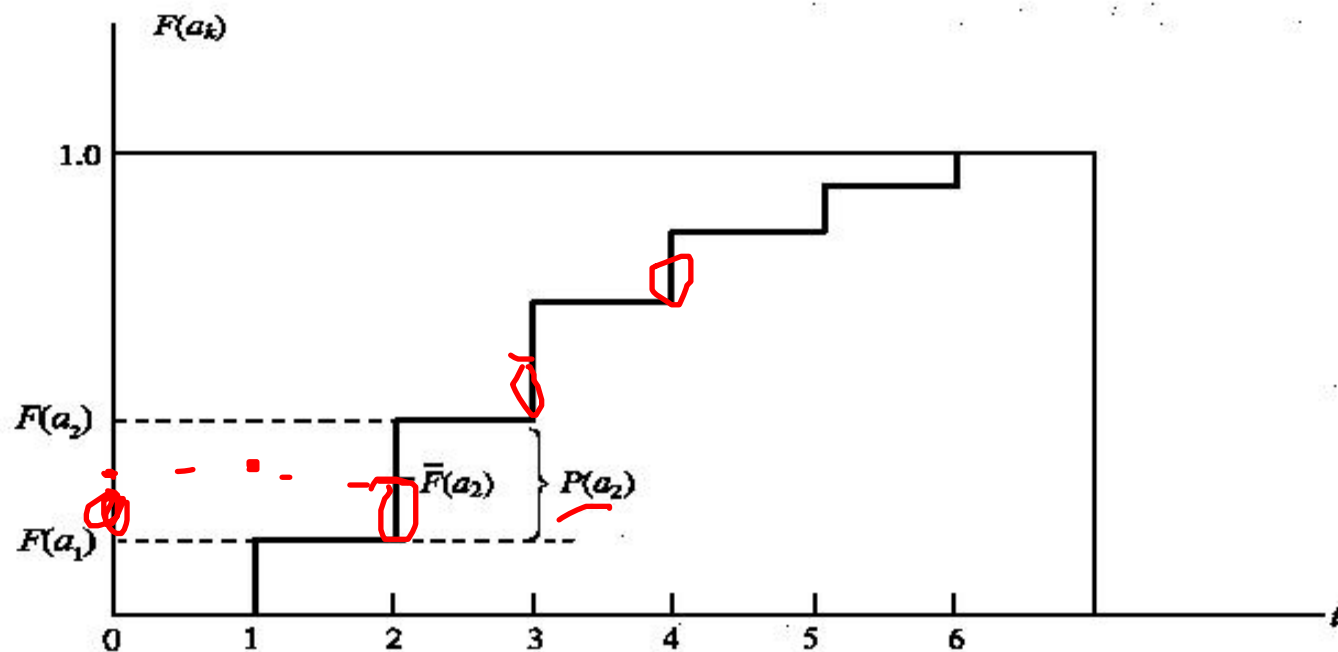


图 8.5 累积分布函数

所有 $P(a) > 0$ 都是正数，所以当 $a \neq b$ 时 $F(a) \neq F(b)$ 。可采用 $\bar{F}(a_k)$ 的数值作为符号 a_k 的码字。 $\bar{F}(a_k)$ 的近似值为 $\left[\bar{F}(a_k) \right]_{l(a_k)}$ ，用这 $l(a_k)$ 位二进制数作为 a_k 的码字。

$$\bar{F}(a_k) - \left[\bar{F}(a_k) \right]_{l(a_k)} < \frac{1}{2^{l(a_k)}} \quad (8.18)$$

若选择

$$l(a_k) = \left\lceil \log \frac{1}{P(a_k)} \right\rceil + 1$$

$$> \log \frac{1}{P(a_k)}$$

(8.19)

得

$$\frac{1}{2^{l(a_k)}} < \frac{P(a_k)}{2} = \bar{F}(a_k) - F(a_k)$$

(8.20)

从 $F(a_k)$ 得区间来看，总区间为 $[0, 1]$ ，若每个信源符号 a_k 所编码得码字对应的区域都没有重叠，那这组码一定满足前缀条件，一定是即时码。

香农—费诺—埃利斯码的平均码长

$$H(S) + 1 \leq \bar{L} < H(S) + 2$$

(8.21)

$$\overline{F}(S) = \overline{F}(a_k) = \sum_{i=1}^{k-1} P(a_i) + \frac{1}{2}P(a_k)$$

$$l(a_k) = \left\lceil \log \frac{1}{P(a_k)} \right\rceil + 1$$

例8.5 离散无记忆信源，它的香农-费诺-埃利斯码如下

信源符号 S	概率函数 $P(s)$	累积分布函数 $F(s)$	$\overline{F}(s)$	$\overline{F}(s)$ 的二进数	码长 $l(s) = \lceil \log \frac{1}{P(s)} \rceil + 1$	码字 W
s_1	0.25	0.25	0.125	0.001	3	001
s_2	0.5	0.75	0.5	0.10	2	10
s_3	0.125	0.875	0.8125	0.1101	4	1101
s_4	0.125	1.0	0.9375	0.1111	4	1111

平均码长为**2.75** 二元码/信源符号。

*8.4 游程码和MH编码（选讲）

前面介绍的编码方法，主要适用于多元信源和无记忆信源。当信源是有记忆时，特别是二元相关信源，就必须对其 N 次扩展信源进行编码才能提高编码效率。

游程编码是一种针对相关信源的有效编码方法，尤其适用于二元相关信源。游程编码已在图文传真、图像传输等实际通信工程技术中得到应用。有时常将游程编码和其他一些编码方法混合使用，能获得更好的压缩效果。

信源输出的字符序列中各种字符连续地重复出现而形成一段一段的字符串，这种字符串的长度称为**游程(Run-Length,RL)**，又称**游程长度**或**游长**。

游程编码（**RLC**）就是将这种字符序列映射成串的字符、串的长度和串的位置的标志序列。知道了串的字符、串的长度和串的位置的标志序列，就可以完全恢复除原来的字符序列。

游程编码不但适用于一维字符序列，也适用于二维字符序列。

1. 二元相关信源的游程编码

二元相关信源，其输出只有两个符号，即“0”和“1”。对应段中的符号个数就是“0”游程长度和“1”游程长度，记为 $L(0)$ 和 $L(1)$ 。二元序列中只有“0”游程和“1”游程，总是交替出现。若规定二元序列总是从“0”游程开始，那么接着第二个必定是“1”游程，然后第三个又是“0”游程…，游程交替出现。这样只需用一个表示串的长度的标志参量就行。信源输出的任意二元序列就可一一对应地映射成交替出现的游程长度的标志序列。表示游程长度的序列，简称游程序列。这种映射是可逆的，是无失真的。例如某二元序列为

00010011111100000001...

可映射成下面的游程序列

31267...

还可以对游程序列采用变长游程编码。一般“0”游程长度和“1”游程长度应分别进行霍夫曼编码。

一般情况下，游程越长，出现的概率就越小；按照霍夫曼编码的规则，概率越小，码长越长，常常在实际应用时，对长游程就不严格按照霍夫曼编码的步骤进行，采用截断处理的方法，将大于一定长度的长游程统一用等长码编码。

2. 二元游程序列的概率特性和编码效率

设二元信源为无记忆信源

$$P[L(0)] = p_0^{L(0)-1} p_1 \quad (8.22)$$

$$P[L(1)] = p_1^{L(1)-1} p_0 \quad (8.23)$$

$$H[L(0)] = \frac{H(p_0)}{p_1} \quad (8.26)$$

$$l_0 = E[L(0)] = \frac{1}{p_1} \quad (8.27)$$

$$H[L(1)] = \frac{H(p_1)}{p_0} \quad (8.28)$$

$$l_1 = E[L(1)] = \frac{1}{p_0} H(p_1) \quad (8.29)$$

$$H(S) = \frac{H[L(0)] + H[L(1)]}{l_0 + l_1} = H(p_0) = H(p_1) \quad (8.30)$$

可见游程变换后信源的信息熵没有改变。

对于有记忆的二元信源，可以证明当信源是一阶、二阶二元马尔可夫信源时，变换后的游程序列是独立序列，游程之间统计独立。 $k > 2$ 时，经变换后的游程序列不再是独立序列，游程之间有依赖关系。一般 k 阶二元马尔可夫信源，经变换后的游程序列将成为 $k-2$ 阶马尔可夫链，相关性要弱。

所以，游程变换能较好地解除或削弱二元序列的相关性，因而再对游程长度进行霍夫曼编码，就可以获得较高的编码效率。

MH编码—修正霍夫曼编码

- 改进的霍夫曼码是文件**传真**中使用的数据压缩技术，是实际使用的信源编码技术。
- **文件传真**是指一般文件、图纸、手写稿、表格、报纸等文件的传真。它是**黑白二值**的，即是二元信源。
- 对于二值灰度的文件传真，每一行总是由若干个白色游程和若干个黑色游程组成。如果测得了这些游程的发生概率，对于**不同的游程长度，按其不同的概率，分配不同的码字，这就是文件传真的游程编码。**

MH 编码—修正霍夫曼编码

- 文件图纸要根据清晰度的要求决定空间扫描分辨率，将文件图纸在空间上离散化。CCITT (ITU) 为了保证传真文件具有足够的清晰度，规定A4幅面 ($210\text{mm} \times 297\text{mm}$) 文件应具有如下分辨率：
 - (1) 有1188或2376条扫描线，即4线/mm或8线/mm。
 - (2) 每条扫描线有1728像素，即8像素/mm。

MH 编码—修正霍夫曼编码

- 文件传真中对文件的黑、白游程长度进行霍夫曼编码。
 - 首先要确切掌握文件信源的概率统计特性，根据这一概率分布制定一套适合该类文件的码表，作为编译码的依据；
 - 其次要对文件信源所有可能的各种概率分布进行变长编码

MH 编码—修正霍夫曼编码

- 采用**截断霍夫曼编码**

不对全部的黑、白游程 2×1728 种制定码表，只将码表划分为两大类型：

结尾码：长度为 $0 \sim 63$ 位

基干码：对长度为64的整数倍的游程

MH 编码—修正霍夫曼编码

- 对黑白游程长度 $l < 64$ 的游程，直接查**结尾码**表进行编码；
- 对于游程长度 l 在 $[64, 1728]$ 范围的游程，则采用**基干码和结尾码联合编码**，即64的整数倍采用基干组合码，余数采用结尾码；
- 码表的总码数为 $2 \times (64 + 1728/64) = 182$ 个。如果不采用上述方法，而是直接对黑白游程分别编码，则码表中码元数为 2×1728 个，可见这种修正霍夫曼编码的压缩率是很大的，而且，该编码的精度基本上满足了大多数系统的需求。

MH 编码—修正霍夫曼编码

- 原邮电部数据所根据汉字的特点，对我国文件传真测试样张进行了统计。在行分辨率为8点/ mm 时，其结果为：
 - 白、黑像素的概率分别为 $P_W=93.3\%$, $P_B=6.7\%$ ，白游程长度 I_W 小于61个像素游程占80%；黑游程长度 I_B 小于6个像素的游程占80%，所以80%的游程编码是无需截断而直接进行霍夫曼编码。
- 从整体看，采用修正霍夫曼编码对性能的影响不是很大。

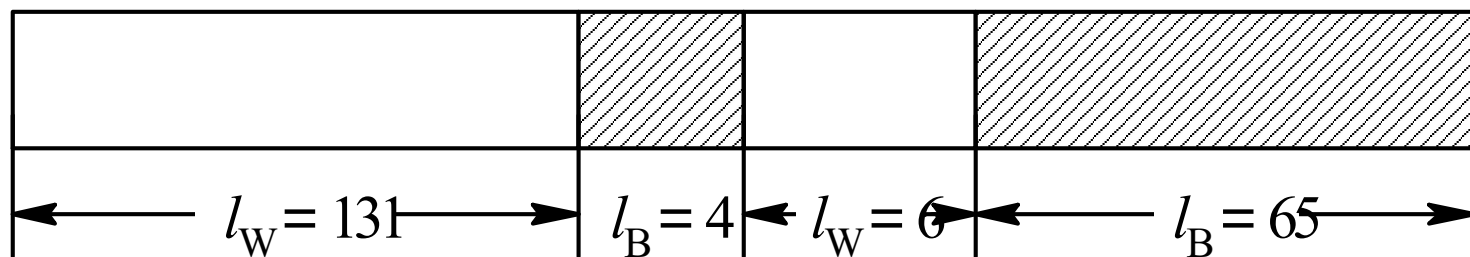
MH编码步骤

- 1) 游程长度在0~63的黑、白游程，码字**直接由结尾码表**(见p287)**查得**。
- 2) 游程长度在64~1728游程，码字由两部分构成，一部分是64整数倍对应的码字，另一部分余数对应结尾码。如白游程为 $129 = 2 \times 64 + 1$ ，则由组合基干码表查得 2×64 对应码字10010，由结尾码表查得余数1对应码字为000111，所以白游程长129对应码字为10010000111。
- 3) 规定**每行都从白游程**开始。若实际出现黑游程开始的话，则在**行首**加上零长度白游程码字。**每行结束**用一个结束码 (EOL)。

MH编码步骤

- 4) 每页文件开始第一个数据前加一个结束码。每页尾连续使用6个结束码表示结尾。
- 5) 每行恢复成**1728**个像素，否则有错。因为霍夫曼码是即时码，所以可以将接收到的二元序列查表译得原二元序列。
- 6) 为了传输时实现同步操作，规定T为每编码行最小传输时间。一般规定 $5s \geq T \geq 20ms$ 。若编码行传输时间小于T，则在结束码之前填以足够的“0”码元（称**填充码**）。

例：现有某一A4文件某行中的一段如图所示。
根据修正霍夫曼码表（MH码表），求出具
体MH码，并求实际压缩比。



解 $l_W = 131 = 2 \times 64 + 3 = 128 + 3$

查得128对应码字为：10010

查得3对应码字为：1000

则131对应MH码为10010：1000 = 100101000

同理, $l_B=4$ MH码为: 011

$l_W=6$ MH码为: 1110

$l_B=65=64+1$

MH码为 0000001111: 010=0000001111010

总编码: 100101000111100000001111010

编码后总码长=5+4+3+4+10+3=29

编码前总码长=131+4+6+65=206

$$\text{实际压缩比} = \frac{\text{编码前总码长}}{\text{编码后总码长}} = \frac{206}{29} = 7.1$$

第五节 算术编码

- 算术编码从全序列出发，考虑符号之间的关系来进行编码。
- 算术编码利用了累积概率的概念。
- 算术码主要的编码方法是计算输入信源符号序列所对应的区间。
- 在编码过程中，每输入一个符号要进行乘法和加法运算，所以称此编码方法为算术编码。

第五节 算术编码

- 设信源符号集 $A=\{a_1, a_2, \dots, a_n\}$, 其相应概率分布为 $P(a_i)$,

$$P(a_i) > 0 \quad (i=1, 2, \dots, n)$$

- 信源符号的累积分布函数

累积分布函数为每台级的下界值, 则其区间为 $[0,1)$ 左闭右开 区间。

$$\underline{F(a_1)=0}$$

$$\underline{F(a_2)=P(a_1)}$$

$$F(a_3)=P(a_1)+P(a_2)$$

...

第五节 算术编码

- 计算二元无记忆信源序列的累积分布函数
 - 初始时：在 $[0,1)$ 区间内由 $F(1)$ 划分成二个子区间 $[0, F(1))$ 和 $[F(1), 1)$ ， $F(1) = P(0)$ 。
 - 子区间 $[0, F(1))$ 的宽度为 $A(0) = P(0)$ ，对应于信源符号“0”；
 - 子区间 $[F(1), 1)$ 的宽度为 $A(1) = P(1)$ ，对应于信源符号“1”；
 - 若输入符号序列的第一个符号为 $s = \text{“0”}$ ，落入 $[0, F(1))$ 区间，得累积分布函数

$$F(s = \text{“0”}) = F(0) = 0$$

第五节 算术编码

输入第二个符号为“1”， $\mathbf{s}=\text{“01”}$

$\mathbf{s}=\text{“01”}$ 所对应的区间是在区间 $[0, F(1))$ 中进行分割；符号序列“00”对应的区间宽度为

$$\underline{A(00)} = A(0)P(0) = P(0)P(0) = \boxed{P(00)}$$

对应的区间为 $[0, F(\mathbf{s}=\text{“01”}))$

符号序列“01”对应的区间宽度为

$$A(01) = A(0)P(1) = P(0)P(1) = P(01) = \underline{A(0) - A(00)}$$

对应的区间为 $[F(\mathbf{s}=\text{“01”}), F(1))$ 。

累积分布函数 $\underline{F(\mathbf{s}=\text{“01”})} = P(00) = P(0)P(0)$

第五节 算术编码

输入第三个符号为“1”，输入序列记作 $s1=“011”$ （若第三个符号为“0”，则为 $s0=“010”$ ）；

输入序列 $s1=“011”$ 对应的区间是对区间
 $[F(s), F(1))$ 进行分割；序列 $s0=“010”$ 对应的区间
宽度为 $A(s=“010”) = A(s=“01”) \cdot P(0) = A(s) \cdot P(0)$

其对应的区间为 $[F(s), F(s) + A(s) \cdot P(0))$ ；

序列 $s1=“011”$ 对应的区间宽度为：

$$A(s=“011”) = A(s) \cdot P(1) = A(s=“01”) - A(s=“010”) = A(s) - A(s0)$$

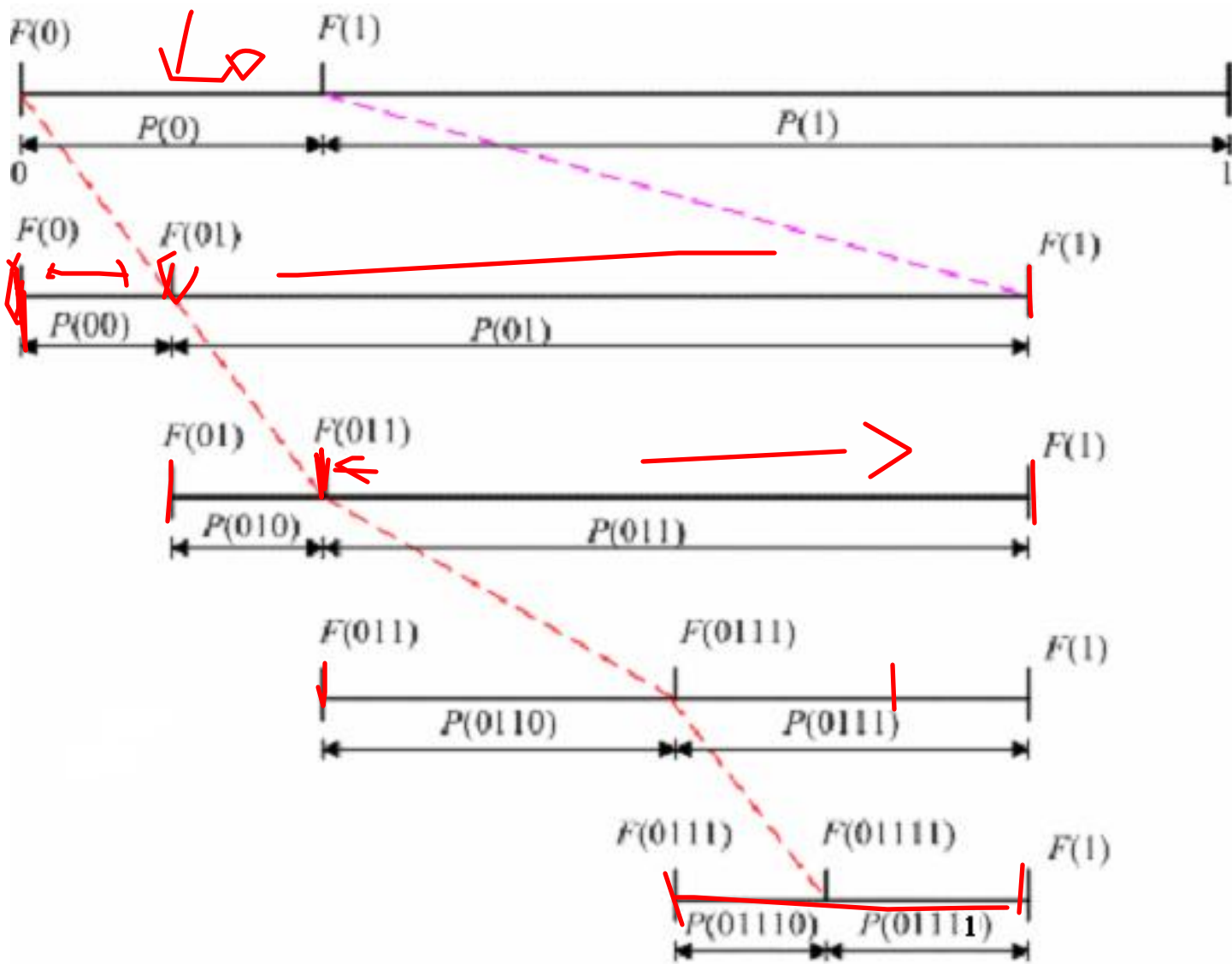
其对应的区间为 $[F(s) + A(s) \cdot P(0), F(1))$ ；

符号序列 $s1=“011”$ 的累积分布函数为

$$F(s1) = F(s) + A(s) \cdot P(0)；$$

若第三个符号输入为“0”，符号序列 $s0=“010”$ 的
区间下界值仍为 $F(s)$ ，得符号序列 $s0=“010”$ 的累
积分布函数为 $F(s0) = F(s)$ 。

第五节 算术编码



第五节 算术编码

■ 归纳

已知前面输入符号序列为 $\mathbf{s}$

- 若接着输入一个符号“0”，序列 $\mathbf{s0}$ 的累积分布函数为：

$$F(\mathbf{s0}) = F(\mathbf{s})$$

对应区间宽度为： $A(\mathbf{s0}) = A(\mathbf{s}) \cdot P(0)$

- 若接着输入的符号是“1”，序列的累积分布函数为：

$$F(\mathbf{s1}) = F(\mathbf{s}) + A(\mathbf{s}) \cdot P(0)$$

对应区间宽度为：

$$A(\mathbf{s1}) = A(\mathbf{s}) \cdot P(1) = A(\mathbf{s}) - A(\mathbf{s0})$$

第五节 算术编码

■ 符号序列对应的区间宽度

$$A(s=\text{"0"})=P(0)$$

$$A(s=\text{"1"})=1 - A(s=\text{"0"})=P(1)$$

$$A(s=\text{"00"})=P(00)=A(0)P(0)=P(0)P(0)$$

$$A(s=\text{"01"})=A(s=\text{"0"}) - A(s=\text{"00"})=P(01)=A(0)P(1)=P(0)P(1)$$

$$A(s=\text{"10"})=P(10)=A(1)P(0)=P(1)P(0)$$

$$A(s=\text{"11"})=A(s=\text{"1"}) - A(s=\text{"10"})=P(11)=A(s=\text{"1"}) \cdot P(1)=P(1)P(1)$$

$$A(s=\text{"010"})=A(s=\text{"01"}) \cdot P(0)=P(01)P(0)=P(010)$$

$$\begin{aligned} A(s=\text{"011"}) &= A(s=\text{"01"}) - A(s=\text{"010"}) = A(s=\text{"01"}) \cdot P(1) \\ &= P(01)P(1)=P(011) \end{aligned}$$

信源符号序列s所对应区间的宽度等于符号序列s的概率 $P(s)$ 。

第五节 算术编码

- 二元信源符号序列的累积分布函数的递推公式

$$F(sr) = F(s) + P(s) \cdot F(r) \quad (r=0,1)$$

s为前面信源符号序列，r为接着输入的符号

$$F(0)=0, \quad F(1)=P(0)$$

$$F(s0) = F(s)$$

$$F(s1) = F(s) + P(s) \cdot F(1) = F(s) + P(s) \cdot P(0)$$

$$A(sr) = P(sr) = P(s) \cdot P(r) \quad (r=0,1)$$

$$A(s0) = P(s0) = P(s) \cdot P(0)$$

$$A(s1) = P(s1) = P(s) \cdot P(1)$$

第五节 算术编码

- 例：已输入二元符号序列为 $\mathbf{s} = \text{"011"}$ ，接着再输入符号为“1”，序列累积分布函数为：

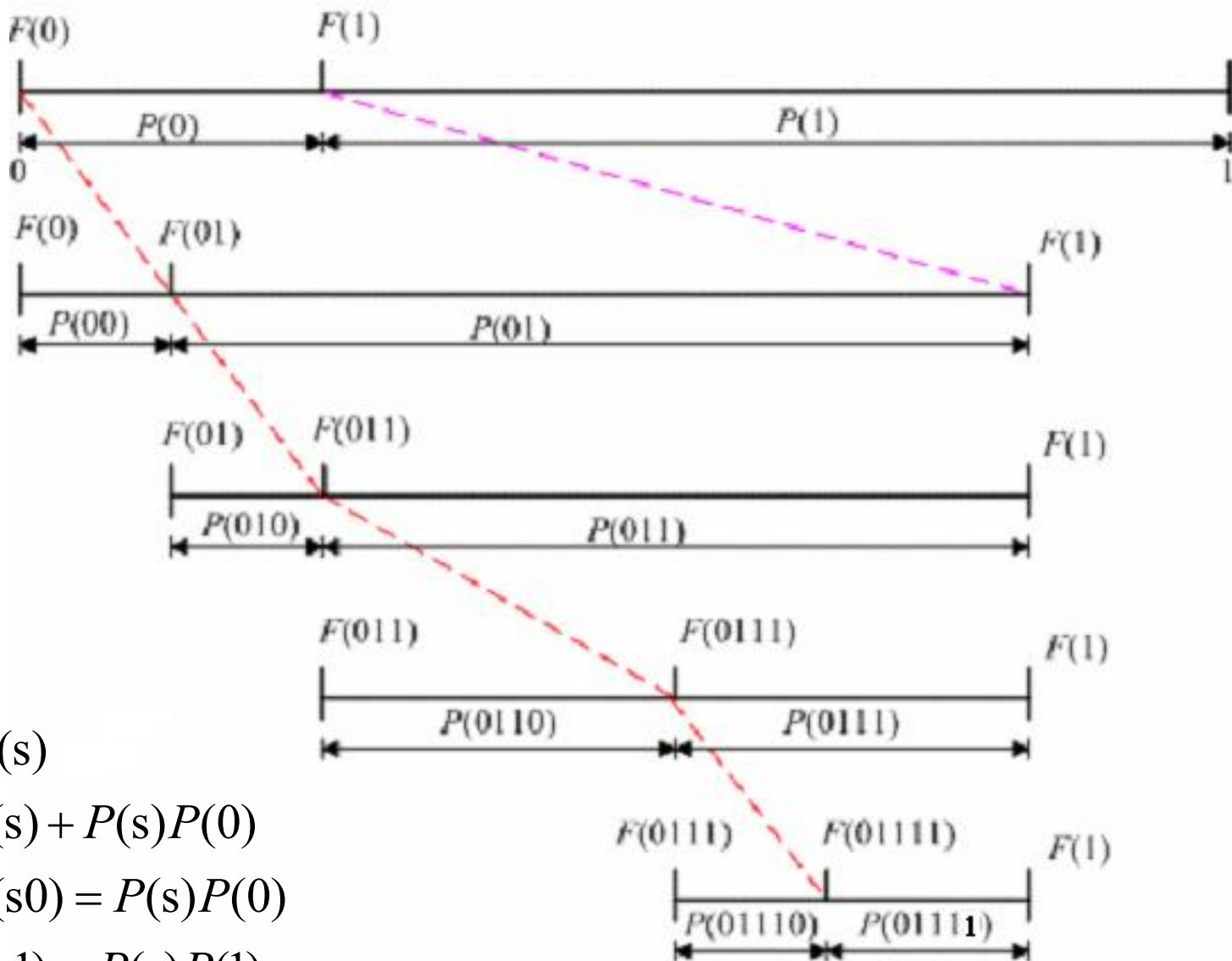
$$\begin{aligned} F(\mathbf{s}1) &= F(0111) = F(\mathbf{s} = \text{"011"}) + P(011) \cdot P(1) \\ &= F(\mathbf{s} = \text{"01"}) + P(01) \cdot P(1) + P(011) \cdot P(1) \\ &= F(\mathbf{s} = \text{"0"}) + P(0) \cdot P(1) + P(01) \cdot P(1) + P(011) \cdot P(1) \\ &= 0 + P(00) + P(010) + P(0110) \end{aligned}$$

对应的区间宽度为

$$A(\mathbf{s}1) = P(\mathbf{s} = \text{"011"}) \cdot P(1) = P(011)P(1) = P(0111)$$

- 上述整个分析过程可用下图描述；
- 只需两个存储器，把 $P(\mathbf{s})$ 和 $F(\mathbf{s})$ 存下来，然后根据输入符号更新两个存储器中的数值。

第五节 算术编码



$$F(s0) = F(s)$$

$$F(s1) = F(s) + P(s)P(0)$$

$$A(s0) = P(s0) = P(s)P(0)$$

$$A(s1) = P(s1) = P(s)P(1)$$

第五节 算术编码

通过信源符号序列的累积分布函数的计算，把区间分割成许多小区间，不同的信源符号序列对应不同的区间为

$[F(\mathbf{s}), F(\mathbf{s}) + P(\mathbf{s}))$ ，取小区间内的一点来代表这序列。

- **编码方法：**将符号序列的累积分布函数写成二进位的小数，取小数点后 k 位，若后面有尾数，就进位到第 k 位，这样得到的一个数 C ，并使 k 满足

$$k = \left\lceil \log_2 \frac{1}{P(\mathbf{s})} \right\rceil \quad (\lceil \rceil \text{代表大于或等于的最小整数})$$

设 $C = 0.z_1z_2 \cdots z_k$ $z_k \in \{0,1\}$ ，得符号序列 s 的码字为 $z_1z_2 \cdots z_k$ 。

- 例 $F(\mathbf{s}) = 0.10110001$ $P(\mathbf{s}) = \frac{1}{7}$ 则 $k = 3$

得 $C = 0.110$ ， s 的码字为110。

第五节 算术编码

■ 编码效率

- 这样选取的数值C，根据二进小数截去尾数的误差得 $C - F(\mathbf{s}) < 1/2^k$

移项有 $F(\mathbf{s}) + 1/2^k > C$

当在k以后没有尾数时 $C = F(\mathbf{s})$

而 $P(\mathbf{s}) \geq 1/2^k$ （根据k的取值可知）

- 信源符号序列对应区间的上界为

$$F(\mathbf{s}) + P(\mathbf{s}) \geq F(\mathbf{s}) + 1/2^k > C$$

- 可见，数值C在区间 $[F(\mathbf{s}), F(\mathbf{s}) + P(\mathbf{s}))$ 内。信源符号序列对应的不同区间（左闭右开）是不重叠的，所以编得的码是即时码。

第五节 算术编码

由于 $k = \left\lceil \log_2 \frac{1}{P(s)} \right\rceil$ 得

$$-\sum_s P(s) \log_2 P(s) \leq \overline{K} = \sum_s P(s) k(s) < -\sum_s P(s) \log_2 p(s) + 1$$

平均每个符号的码长为

$$\frac{H(S^L)}{L} \leq \frac{\overline{K}}{L} \leq \frac{H(S^L)}{L} + \frac{1}{L}, \quad \text{若信源是无记忆, } H(S^L) = LH(S)$$

$$H(S) \leq \frac{\overline{K}}{L} = R < H(S) + \frac{1}{L} \quad \left(R = \frac{\overline{K}}{L} \log_2 m \right)$$

算术编码的编码效率很高，当信源符号序列很长（L很大）时，平均码长接近信源熵

第五节 算术编码

译码就是一系列比较过程，每一步比较 $C - F(\mathbf{s})$ 与 $P(\mathbf{s})P(0)$ 。

$$F(\mathbf{s}0) = F(\mathbf{s}) \quad F(\mathbf{s}1) = F(\mathbf{s}) + P(\mathbf{s})P(0)$$

- $\mathbf{s}$ 为前面已译出的序列串；
- $P(\mathbf{s})$ 是序列串 $\mathbf{s}$ 对应的宽度；
- $F(\mathbf{s})$ 是序列串 $\mathbf{s}$ 的累积分布函数，即为 $\mathbf{s}$ 对应区间的下界；
- $P(\mathbf{s})P(0)$ 是此区间内下一个输入为符号“0”所占的子区间宽度；

若 $C - F(\mathbf{s}) < P(\mathbf{s})P(0)$ 则输出符号为“0”；

若 $C - F(\mathbf{s}) > P(\mathbf{s})P(0)$ 则输出符号为“1”。

算术编码算法过程

- ① 设空信源序列为 λ , 置 $P(\lambda)=1, F(\lambda)=0$;
- ② 对 $i=1, 2 \cdots n$, 在二进制数字系统中按序执行下列运算:

$$F(x^i) = F(x^{i-1}) + p(x^{i-1}).c(x_i)$$

$$c(x) = \begin{cases} 0 & x = 0 \\ p(0) & x = 1 \end{cases}$$

$$p(x^i) = p(x^{i-1}).p(x_i)$$

计算

$$k(x^n) = \left\lceil \log \frac{1}{p(x^n)} \right\rceil$$

与序列对应的码字就是把 $F(x^n)$ 按二进制小数展开取小数点后的 $k(x^n)$ 位, 若后面有尾数则进位到 $k(x^n)$ 位。

算术编码的译码过程

(1) 置 $C_0 = C(x^n)$

(2) 对于 $i=1, 2, \dots$ 年, 在二进制系统中执行下列运算

$$\text{a) } z_i = \begin{cases} 1 & p(z^{i-1}) \cdot p(0) \leq C_{i-1} \\ 0 & p(z^{i-1}) \cdot p(0) > C_{i-1} \end{cases}$$

$$\text{b) } C_i = C_{i-1} - p(z^{i-1}) \cdot c(z_i)$$

$$c(z) = \begin{cases} 0 & z=0 \\ p(0) & z=1 \end{cases}$$

$$\text{c) } p(z^i) = p(z^{i-1}) \cdot p(z_i)$$

(3) 输出译码 $z^n = z_1 z_2 \cdots z_n$

第五节 算术编码

例：设二元无记忆信源 $S=\{0,1\}$ ，其 $P(0)=1/4$ ，

$P(1)=3/4$ 。对二元序列 11111100 做算术编码

解： $P(\mathbf{s}=11111100)=P^2(0)P^6(1)=(3/4)^6(1/4)^2$

$$k = \left\lceil \log_2 \frac{1}{P(\mathbf{s})} \right\rceil = 7$$

$$F(\mathbf{s}r) = F(\mathbf{s}) + P(\mathbf{s}) \cdot F(r)$$

$$F(\mathbf{s}0) = F(\mathbf{s}) \quad F(\mathbf{s}1) = F(\mathbf{s}) + P(\mathbf{s}) \cdot F(1) = F(\mathbf{s}) + P(\mathbf{s}) \cdot P(0)$$

$$\begin{aligned} F(\mathbf{s}) &= P(0) + P(1)P(0) + P(1)^2P(0) + P(1)^3P(0) + P(1)^4P(0) + P(1)^5P(0) \\ &= 0.82202 = 0.110100100111 \end{aligned}$$

得 $C=0.1101010$ 得 $\mathbf{s}$ 的码字为 1101010。

编码效率

$$\eta = \frac{H(s)}{R} = \frac{H(s)}{K/L} = \frac{0.811}{7/8} = 92\%$$

算术编码特点

- 1) 算术编码有基于概率统计的固定模式，也有相对灵活的自适应模式，**自适应模式的工作方式是：为各个符号设定相同的概率初始值，然后根据出现的符号做相应的改变。**
- 2) 自适应模式适用于不进行概率统计的场合。
- 3) 当信号源符号的出现概率接近时，算术编码的效率高于霍夫曼编码。
- 4) 算术编码的实现相应地比霍夫曼编码复杂，但在图像测试中表明，算术编码效率比霍夫曼编码效率高 5 % 左右。

香农-费诺-埃利斯码和算术编码的不同

(1) 定义的累积分布函数不同。香农 - 费诺 - 埃利斯码的累积分布函数 $F(a_k)$ 是每台级的上界值。而算术编码的累积分布函数是每台级的下界值。

(2) 信源符号对应选取的码字不同。当确定信源符号 a_k 的码字位数 l_k 后, 香农 - 费诺 - 埃利斯码是选取 $\bar{F}(a_k)$ 的二进位小数后 l_k 位(截去后面尾数)的二进数作为码字 C 。而算术编码是选取 $F'(a_k)$ 的二进位小数后 l_k 位(有尾数时就进位到 l_k 位)的二进数作为码字 C' 。

另外, 正是因为这两种编码所定义的累积分布函数不同, 所以他们选取的码字对应的区间也不同。

对于香农 - 费诺 - 埃利斯码, 其 $F(a_k)$ 的总区间是 $(0, 1]$, 左开右闭区间。编码是将区间 $(0, 1]$ 分割成许多小区间, 信源符号 a_k 所编码字的数值对应的区间是 $(F(a_{k-1}), \bar{F}(a_k)]$ (左开右闭), 码字的数值不可能落在 $F(a_{k-1})$ 上。并且有 $\bar{F}(a_k) - C < 2^{-l_k}$, 即码字靠近每台级中点, 或落在中点上。

对于算术编码, 其 $F'(a_k)$ 的总区间是 $[0, 1)$, 左闭右开区间。编码是将区间 $[0, 1)$ 分割成许多小区间, 信源符号 a_k 所编码字数值对应的区间是 $[F'(a_k), F'(a_k) + P(a_k)]$ (左闭右开), 码字的数值有可能落在 $F'(a_k)$ 上。(注意: $F'(a_k) = F(a_{k-1})$ 。)

并且有 $C' - F'(a_k) < 2^{-l_k}$, 即码字靠近每台级 $F'(a_k)$, 或落在此边界点 $F'(a_k)$ 上。

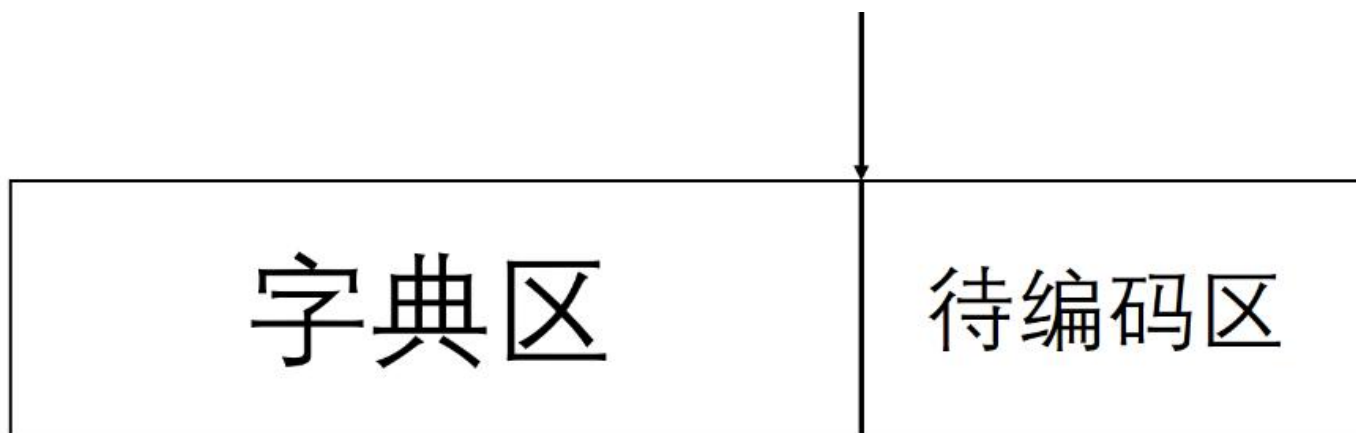
第六节 字典码—LZ编码

- 前面介绍的各种编码方法，都需要知道信源的分布，称为**统计编码**，在实际应用中存在困难。
- 如果一种编码方法与信源的统计特性无关，只要满足 $R > H(X)$ ，当编码长度充分大时，错误概率可以任意的小，这样的编码称为**通用信源编码**。
- 1965 苏联数学家kolmogolov提出了利用信源序列的**结构特性**来编码。
- 1977 以色列学者J.Ziv, A.Lempel 提出了更快捷的通用压缩算法LZ77，在之后出现了一系列的改进算法，如LZ78, LZW等，统称为LZ系列编码算法。

- LZ算法将字典技术应用于通用数据压缩领域。
- LZ算法可以逼近信息熵的极限。

第六节 字典码—LZ77

- 在LZ-77算法中，整体思路是，我们先手动设定一个窗口，这个窗口包含两个部分，其中之一是位于左半部分的字典区域，另一个是位于窗口右端的待编码区。



- 用三元标识(K,l,d)给数据编码，K是窗内从尾**自右向左**搜索的字符位数，称为移位数，l是窗内可与窗外有相同符号的字符串的长，称为匹配串长度，d是窗外已找到匹配的串的后面串的首字符。

第六节 字典码—LZ77

2. 71828 1828459045...

第1步。开始时,窗内是空的,要输入当前位置的2,窗内没有与2匹配的串,故 K, l 皆为0,把2编码为 $(0, 0, 2)$,输入2。下面输入的数据为.718也与2的情况相同,窗内只有刚输入的2,没有与.718分别相匹配的串,故只能将.718分别编码为 $(0, 0, .)$, $(0, 0, 7)$, $(0, 0, 1)$, $(0, 0, 8)$,分别将其输入窗内。

第2步。现窗内已有2.718存入,当前位置是e的小数第4位2:

2. 71828182845904523536...

在窗内从尾由右向左搜索到第5位处(包括小数点),有窗内左边的第5个数字2与当前位置2相同,即匹配串长 l 为1,当前位置2后面的串的首字符为8,得 $K=5, l=1$,故编码为 $(5, 1, 8)$,表示窗内字典从右向左第5位有一个长为1的匹配串“2”和当前位置的数据串“2”相匹配,后面串的首字符为8,并将28同时输入。

第 3 步。现窗内字典为 2.71828, 当前输入数据为 1:

2.71828182845904523536...

编码器在窗内从尾自右向左搜索, 移动 4 位, 搜索到长度 $l=4$ 的与当前位置相匹配的串: 1828, 下面串的首位为 4, 故编码为 (4, 4, 4)。将 18284 输入窗内。

第 4 步。现窗内字典和当前位置如下:

2.71828182845904523536...

由于字典内没有与 5, 9, 0 相匹配的串, 故均是 0 移位, 0 匹配串长的无匹配字符, 编码为 (0, 0, 5), (0, 0, 9), (0, 0, 0), 并将 5, 9, 0 输入窗内。

第 5 步。现窗内字典和当前位置如下:

2.71828182845904523536...

在窗内搜索到第 4 位, 有“45”与当前位置 45 匹配, 串长 $l=2$, 后续段首字符为 2。

注意点:

(1) 若字典内搜索到两个以上的匹配串,应选匹配长度 l 大的。若 l 值相同,若此时还将对编码后的标识序列再进行二次编码,则取移位数小的串的 K 值(这是改进的 LZH 码);若无二次编码,选移位大的串的 K 值。

(2) 若出现下述情形:

able eeeeeth...

则可编码为 $(1, 5, t)$, 将 *eeeeet* 都输入窗内, 这里必须 *eeeeet* 都在待编码的缓存器内。

LZ-77 码的译码器要有一个与编码器字典窗口等长的缓存器, 可以边译码, 边建立译码字典, 即编码字典不用传送, 译码器可自动生成。这种码隐含了假设条件: 输入数据的产生模式是相距很近的。

练习 **aacaacabcababaaac**

第六节 字典码—LZ78

- 算法基于对信源输出序列的**相异分解**。
 - 把长度为n的信源序列分解为**字符片段**，使得每个字符片段都是前面没有出现过的，而且分段尽可能短。
 - 除了单字符的片段，每个分段的**前缀**必定是前面某个出现过的字符片段。字符片段就可以由它的**最后符号位**和相应的**前缀**唯一确定。
 - **编码**由这个**前缀对应的段号**加上最后的**符号位编码**来确定。



K



d

第六节 字典码—LZ78

- 设信源序列： $S = s_1 s_2 \dots s_i \dots s_n$ ，
 $s_i \in \{u_0, u_1, u_2, \dots, u_{m-1}\}$ ，
- 分段规则是使连着的信源符号尽可能的少，并保证各段都不相同。设信源序列按此规则分段的结果为 $y_1, y_2, y_3, \dots, y_c$ ， c ：序列段数。
- 将**段号**和**信源符号**分别进行编码，若组成二元码，
 - **段号**所需码长： $l_1 = \lceil \log c \rceil$ ；
 - 每个**信源符号**所需码长： $l_2 = \lceil \log m \rceil$

例 信源符号集为 $U = \{a_0=00, a_1=01, a_2=10, a_3=11\}$,
求信源序列 $S = a_0 a_0 a_2 a_3 a_1 a_1 a_0 a_0 a_0 a_3 a_2$ 的 LZ78 编码。

- 根据上述分段原则，信源序列分为7段：

$a_0, a_0 a_2, a_3, a_1, a_1 a_0, a_0 a_0, a_3 a_2$

段号	1	2	3	4	5	6	7
符号	a_0	$a_0 a_2$	a_3	a_1	$a_1 a_0$	$a_0 a_0$	$a_3 a_2$

- 则信源序列对应编码序列为：

000 00, 001 10, 000 11, 000 01, 100 00,
001 00, 011 10

前缀的段号

(K, d)

信源符号

第六节 字典码—LZW

由韦尔奇在 1984 年开发的 LZW 算法,是 LZ 系列码中应用最广、变形最多的。它的标识只有一项,即指向字典的指针,这是它与 LZ-77、LZ-78 的一个主要不同点。为实现简化标识,它的编码的思想与前述的算法有很大的不同。LZ 系列算法的共同点是分解输入流,使其成为长度各异的“短语”,并把它们存入“短语字典”,并给每个“短语”赋予一个码字(经常是短语的字典顺序号,这最简捷)。只要短语的码字长度短于短语的长度,就达到了压缩的目的。而 LZW 编码算法则是先建立初始字典,再分解输入流为短语词条,这个短语若不在初始字典内,就将其存入字典,这些新词条和初始字典共同构成编码器的字典。而初始字典可由信源符号集构成,每个符号是一个词条。更一般的,是将扩展的 ASCII 码存入初始字典,使其成为字典的前 256 项,即 0~255 项。这样的初始化字典,在应用中就足够大了。

LZW 码的编码原理是:先建立初始化字典,然后将待编码的输入数据流分解成“短语词条”。编码器要逐个输入字符,并累积串联成一个字符串,即“短语词条” I 。若 I 是字典中已有的词条,就输入下一个字符 x ,形成新词条 Ix 。当 I 在字典内,而 Ix 不在字典内时,编码器首先输出指向字典内词条 I 的指针(即 I 的相应码字);再将 Ix 作为新词条存入字典,并为其确定顺序号;然后把 x 赋值给 I ,当做新词条的首字符。重复上述过程,直到输入流都处理完为止。下面还用实例来说明 LZW 码编码过程。

【例 8.10】 设输入序列为

ababcbabccc

- (1) 先建初始化字典,此处只需将信源符号 a, b, c 预置为字典的前 3 项;
- (2) 将首字符 a 预置为 I ,即 $I=a$,搜索后知 I 在字典内,那么继续输入序列第二项 b ,即有 $Ix=ab$,搜索后知 Ix 不在字典内。则编码器先输出指向字典词条 $I=a$ 的指针即 a 相应的码字 1,再把 $Ix=ab$ 作为新词条存入字典,并编码得码字为 4。再将 x 赋值给 I ,即此时 $I=b$,当作新词条的首字符重复上述做法,得到编码表,如表 8.13 所示。

表 8.13 【例 8.10】LZW 码的编码表

	码字	词条 <i>I</i>	新词条	输出码
初始字典	1	<i>a</i>		
	2	<i>b</i>		
	3	<i>c</i>		
	4	<i>a</i>	No	1
		<i>ab</i>	Yes	
		<i>b</i>	N	
	5	<i>ba</i>	Y	2
		<i>a</i>	N	
		<i>ab</i>	N	
	6	<i>abc</i>	Y	4
		<i>c</i>	N	
		<i>cb</i>	Y	
	7	<i>b</i>	N	3
		<i>ba</i>	N	
		<i>bab</i>	Y	
	8	<i>b</i>	N	5
		<i>bc</i>	Y	
		<i>c</i>	N	
	9	<i>cc</i>	Y	2
		<i>c</i>	N	
		<i>cc, eof</i>	N	
	10	<i>c</i>	Y	3
		<i>c</i>	N	
		<i>cc, eof</i>	N	10, eof

LZW 的解码器也需首先建立初始化字典:字典或是由信源符号集构成(此时,编码器要传送初始字典),或是由扩展的 ASCII 码构成(此时不用传送初始字典)。解码的第 1 步,输入第 1 个码字(即第 1 个指针),并从字典中取回一个词条 I ,并将 I 输出,同时将 Ix 存入解码字典中。但此时, x 是未知的, x 将是下一个从字典中读取的词条的首个字符。再输入下一个指针(即码字),又从字典中取回词条 J ,将其输出,并把 J 的首字符赋予上一步存入字典的词条 Ix 的 x ,则此时, Ix 已完全确定。重复上述过程,则自动重建了译码表,并将译码输出。

仍以例 8.10 题来说明解码过程,其接收码字为 1,2,4,3,5,2,3,10,eof;

(1)先建解码的初始化字典,此处是信源符号 a, b, c ,同于编码器;

(2)读入第一个接收码字(即第一个指针)“1”,从字典中取出为 a ,将符号 a 输出,同时将 $Ix = ax$ 存入字典作为码字 4;

(3)再读入下一个接收码字“2”,从字典中取出为 b ,一方面将 b 输出,同时确定上一步中 $x = b$,得字典中码字 4 即 $Ix = ab$;然后将 b 置于 I ,又存入新词条 $Ix = bx$ 作为码字 5;

(4)读入下一个接收码字“4”,从字典中取出为 ab ,输出符号 ab ;一方面确定上一步中 $x = a$ (读取词条 ab 中的首个字符),得码字 5 为 ba ;一方面又将 ab 置于 I ,又存入新词条 $Ix = abx$ 作为码字 6;其中 x 将由下一个输入码字从字典中读取的词条的首个字符决定;

重复上述过程,则重建了译码表(同编码表 8.13),又译码输出序列为 $ababcbabccc$ 。

第六节 字典码—LZ编码

- LZ的译码过程也很简单，不需要传输字典，可以一边译码，一边建立字典。
- LZ编码在信源序列较短时，性能会变坏，但序列增长时，编码效率就会提高。**平均码长也已经被证明逼近信源熵。**
- 应用
 - 在Unix系统中最先出现了使用该算法的compress程序，成为Unix世界的压缩标准，MS-DOS环境下的ARC程序，以及80年代后的著名压缩工具LHarc，ARJ等都运用了LZ算法。
 - 目前，LZ系列编码算法几乎垄断了整个通用数据压缩领域，Winzip，Winrar，gzip以及ZIP，GIF，PNG等文件格式都是LZ系列算法的受益者。

信源编码总结

- 6种信源编码：香农编码、费诺编码、霍夫曼编码、游程编码、算术编码、LZ编码。
- 离散信源变长码优缺点
 - 优点：提高编码效率；
 - 缺点：需要大量缓冲设备来存储这些变长码，然后再以恒定的码率进行传送；在传输的过程中如果出现了误码，容易引起错误扩散，所以要求有优质的信道。
- 有时为了得到较高的编码效率，先采用某种正交变换，解除或减弱信源符号间的相关性，然后再进行信源编码；有时则利用信源符号间的相关性直接编码。

8.1 求概率分布为 $(1/3, 1/5, 1/5, 2/15, 2/15)$ 信源的二元霍夫曼码。讨论此码对于概率分布为 $(1/5, 1/5, 1/5, 1/5, 1/5)$ 的信源也是最佳二元码。

8.4 信源空间为

$$\begin{bmatrix} S \\ P(s) \end{bmatrix} = \begin{bmatrix} s_1, & s_2, & s_3, & s_4, & s_5, & s_6, & s_7, & s_8 \\ 0.4, & 0.2, & 0.1, & 0.1, & 0.05, & 0.05, & 0.05, & 0.05 \end{bmatrix}$$

码符号为 $X = \{0, 1, 2\}$, 试构造一种三元的紧致码。

8.15 已知二元信源 $\{0, 1\}$, 其 $p_0 = \frac{1}{8}, p_1 = \frac{7}{8}$, 试用式(8.56)求出下列序列算术编码的码字, 并计算此序列的平均码长和编码效率

11111110111110

8.16 对输入数据流 $a_0 a_1 a_2 a_3 a_0 a_1 a_2 a_0 a_1 a_2 a_2 a_0 a_1 a_2 a_4 a_0 a_1 a_2 a_5$ 分别用 LZ-77 算法, LZ-78 算法, LZW 算法进行编码, 并计算各种方法的压缩率。

